SpotLet

Spot it. Let it. Done.

PROJECT BLUEPRINT & SOFTWARE ARCHITECTURE SPECIFICATION

Version 1.0 June 2026 Production-Grade Specification

Table of Contents

19 AI Context Summary

01

1.1 Project Identity

SpotLet | Map-Centric | Peer-to-Peer

Spot it. Let it. Done.

1.2 Project Purpose

SpotLet is a map-centric, peer-to-peer rental discovery platform designed to eliminate the friction, high brokerage fees, and location inaccuracy that plague the Indian residential rental market. It serves as a direct bridge between property owners and house seekers.

1.3 Target Users

- Individuals moving to tier-1 (e.g., Bengaluru, Mumbai, NCR) and tier-2/3 cities in India who need to find, verify, and secure accommodations (rooms, PGs, apartments) quickly and without paying high broker fees.
- Homeowners who want to list their residential properties directly, manage leads, and communicate with potential tenants without paying listing fees or dealing with intermediaries.

1.4 Real-World Problems Solved

1. Exorbitant Brokerage Fees: Seekers are routinely forced to pay 1 to 2 months' rent as non-refundable brokerage fees.
2. Opaque and Inaccurate Location Data: Existing listing websites display vague descriptions like "near metro station," which can turn out to be miles away. SpotLet enforces exact GPS pin placements on a map.
3. Stale and Outdated Listings: Standard classified platforms are cluttered with listings that were rented months ago.
4. Complex Onboarding Processes: Most rental portals require lengthy registration forms, passwords, and profile details. SpotLet uses passwordless SMS OTP login.

5. Lack of Direct Communication Channels: Platforms often hide owner details behind paywalls. SpotLet offers single-tap deep links to call or message via WhatsApp directly.

1.5 Core Product Value Proposition

- Instantly display available spaces as clickable price bubbles on a premium dark map.
- Direct communication via deep links (Phone, WhatsApp) with zero mediation.
- Excludes middlemen from the leasing transaction.
- SMS phone verification (E.164) that validates real identity in seconds.
- Drop a pin, type the address, select amenities, upload images, and publish in under 2 minutes.

1.6 Expected Final Outcome

A high-performance, responsive cross-platform application (Web, Android, iOS) that allows users to spot available rentals immediately on a map, filter criteria dynamically, and initiate contact with the owner in a single interaction.

02

2.1 Long-Term Vision

To build the most trusted, map-native decentralized real estate network in India, expanding from residential rooms and PGs to commercial, co-working, and vacation rentals. The long-term goal is to make physical property hunting completely obsolete by combining highly precise spatial coordinates with verified owner documentation.

2.2 Scalability Goals

- Scaling to support millions of active listings with sub-second map cluster rendering.
- Supporting hundreds of thousands of concurrent map search requests using spatial databases (PostGIS).
- Optimizing photo uploads using edge-accelerated CDN resizing.

2.3 Monetization Models

1. Landlord Premium Visibility: Paid features to highlight listings or pin them at the top of searches in high-demand zones.
2. Tenant Screening & Verification SaaS: Offering automated background checks, Aadhaar integration, and tenant rating services for a nominal fee.
3. Rental Agreement & Digital Escrow: Monetizing secure in-app lease agreements and rental deposit escrows with transaction processing fees.
4. Property Management Tools: A subscription service for landlords with multiple units to track rent collections, issues, and tenancy cycles.

2.4 Competitive Advantage

SpotLet differentiates itself from competitors (like NoBroker, MagicBricks, Housing.com) by:

- Prioritizing map interactions over lists.

- Retaining a strictly dark-themed, premium interface that eliminates visual clutter.
- Minimizing onboarding barriers with passwordless OTP.
- Providing a dedicated web experience using an iframe Leaflet map fallback, avoiding React Native map compilation failures on web.

03

Phone OTP Authentication CORE

Purpose: Frictionless, passwordless registration and login.

User Flow: User enters phone number → Receives 6-digit verification code → Enters code → Session starts.

- Frontend: Validates input structure using E.164 format. Displays countdown timers and error dialogs for incorrect inputs.
- Backend: Triggers `supabase.auth.signInWithOtp` sending an SMS code to the client.
- Database: Creates a record in `auth.users` on successful validation.
- Validation: Phone number must be exactly 10 digits (excluding country code), formatted to E.164 (e.g., `+91XXXXXXXXXX`).
- Error Handling: Catches rate-limiting errors (1 req/min) and incorrect verification code errors.
- Security: Employs Supabase's built-in SMS verification security controls.

Home Feed with Search & Filters DISCOVERY

Purpose: Discovery of properties via scrollable feeds, keyword search, and filters.

User Flow: User lands on feed → Scrolls through cards → Enters keyword or adjusts filters → Feed updates immediately.

- Frontend: Instant client-side list filtering utilizing React state effects. Pull-to-refresh resets local list.
- Backend: Fetches all available records from `properties` table, sorted by date.
- Database: `SELECT * FROM properties WHERE available = true ORDER BY created_at DESC`
- Validation: Chained filters evaluate: `type`, `furnished`, `for_whom`, and query matches on `title`, `address`, `description`.
- Error Handling: Displays network error states with connection retries if fetch fails.

- **Security:** SQL data exposure restricted to rows marked `available = true`.

Live Map Discovery DISCOVERY

Purpose: Visual exploration of properties near user's coordinate.

User Flow: User selects Map → Map zooms to current location → Click bubbles to highlight details in carousel → Tap details.

- **Frontend (Mobile):** Uses `react-native-maps` with native Google Maps.
- **Frontend (Web):** Uses an embedded `iframe` containing Leaflet and CartoDB Dark Matter tiles.
- **Backend:** Syncs active markers from database coordinate data.
- **Database:** Fetches `latitude`, `longitude`, `rent`, `id` from `properties`.
- **Error Handling:** Handles location permission denials by centering map on Koramangala, Bengaluru.
- **Security:** User coordinates are processed locally and not saved to database tables.

Property Creation Form OWNER

Purpose: Enables landlords to submit property details.

User Flow: User inputs details → Drags map pin for coordinates → Picks and uploads photos → Clicks submit → Success redirect.

- **Frontend:** Address text geocoding (1.5s debounce). Interactive mini-map pin. Uploads to Cloudinary via REST API.
- **Backend:** Inserts record with user's ID as `owner_id`.
- **Database:** SQL `INSERT` into the `properties` table.
- **Validation:** Rent and deposit must be numeric; title, address, phone numbers are required.
- **Error Handling:** Rollbacks on failure. Fallback to Unsplash photo placeholders if upload fails.

- **Security:** Requires active session token. Enforces user ownership constraints.

Bookmarking (Save / Unsave) ENGAGEMENT

Purpose: Persists selected listings for future retrieval.

User Flow: User clicks heart icon → Heart turns red → Property saved → Viewable in "Saved" tab.

- **Frontend:** Optimistic UI updates: immediately changes heart state locally, reverting on backend failure.
- **Backend:** Adds or removes records in the join table.
- **Database:** `INSERT` or `DELETE` on `saved_properties` table.
- **Validation:** Authenticated users only. Uniqueness constraint `UNIQUE (user_id, property_id)`.
- **Error Handling:** Shows alerts on failures, restoring local heart state.
- **Security:** RLS policies prevent viewing or modifying other users' bookmarks.

04

4.1 User Types

Seeker (Guest / Authenticated)

- **Guest:** Can browse feed, map, and open details modal. Cannot save properties, post listings, or view profiles.
- **Authenticated:** Can search, view exact location pins, save properties, and initiate WhatsApp/Call links.

Owner (Authenticated Landlord)

- Has all Seeker permissions.
- Can create listings, upload media, and manage their posted properties.

Administrator (Planned)

- Full bypass read/write rights.
- Access to admin portals for listing moderation, reports, and spam flags.

4.2 Route Protection Matrix

Route	Role Requirements	Behavior on Failure
`/(tabs)/index`	Anonymous / Authenticated	None (Public Search)
`/(tabs)/map`	Anonymous / Authenticated	None (Public Discover)

Route	Role Requirements	Behavior on Failure
`/(tabs)/add`	Authenticated (`user_id` required)	Redirects to `/auth/login`
`/(tabs)/saved`	Authenticated (`user_id` required)	Renders "Sign In Required" fallback
`/(tabs)/profile`	Authenticated (`user_id` required)	Redirects to `/auth/login`
`/auth/*`	Anonymous Only	Redirects to `/(tabs)`

4.3 Session Management

- Native mobile uses `@react-native-async-storage/async-storage`. Web uses `localStorage`.
- Automated via Supabase SDK's listener `onAuthStateChange`.
- PKCE (Authorization Code Exchange) to protect token leakage.

05

Frontend Core

Technology	Version	Purpose
React	19.1.0	UI rendering engine
React Native	0.81.5	Cross-platform native bridge
React DOM	19.1.0	Web rendering target
Expo SDK	~54.0.35	Cross-platform framework
Expo Router	~6.0.24	File-based navigation

UI & Native Libraries

Library	Version	Purpose
React Native Paper	^5.11.0	Material Design 3 component library
React Native Maps	1.20.1	Google Maps SDK (mobile only)
Expo Image Picker	~17.0.11	Media gallery access
Expo Location	~19.0.8	GPS coordinate access
React Native Reanimated	~4.1.1	Animations engine
Leaflet	1.9.4	Web map fallback (iframe)

Backend & Infrastructure

Service	Version	Purpose
Supabase (PostgreSQL)	15.x	Database, Auth, PostgREST API
@supabase/supabase-js	^2.44.0	Client SDK
Cloudinary REST API	v1_1	Media CDN & image uploads

5.1 Technology Rationale

- Single cross-platform codebase for iOS, Android, and Web with native performance.
- PostgreSQL, RLS security, and JWT-based Phone SMS OTP out of the box.
- Enforces Material Design 3 guidelines for consistent styling.
- Lightweight REST interface without heavy native SDK bindings.
- Prevents app bundler crashes on web by bypassing Google Maps native compilation.

6.1 Folder Structure

```
SpotLet/
├── app/                                  # Expo Router file-based directory
│   ├── _layout.tsx                       # Root wrapper: Providers & Global Routing
│   ├── index.tsx                         # Landing Auth Guard
│   ├── (tabs)/                           # Authenticated Router Zone
│   │   ├── _layout.tsx                   # Platform-aware Tab navigator
│   │   ├── index.tsx                     # Home Feed
│   │   ├── map.tsx                       # Live Map
│   │   ├── add.tsx                       # Add Property Listing Form
│   │   ├── saved.tsx                     # Saved/Bookmarked Listings
│   │   └── profile.tsx                   # User Profile
│   └── auth/                             # Guest Router Zone
│       ├── _layout.tsx                   # Auth slot layout
│       ├── index.tsx                     # Redirects to login
│       └── login.tsx                     # Phone Input & OTP screen
├── assets/                               # Adaptive Icons, Splashes, Favicons
├── components/                           # Shared Components
│   ├── CustomMap.tsx                     # Native maps wrapper (mobile)
│   ├── CustomMap.web.tsx                 # Leaflet iframe wrapper (web)
│   └── PropertyDetailsModal.tsx          # Property details sheet/modal
├── contexts/
│   └── AuthContext.tsx                   # Session management provider
├── lib/
│   ├── supabase.ts                       # Supabase init & DB helpers
│   ├── cloudinary.ts                     # Cloudinary REST API helper
│   └── polyfills.ts                      # Hermes DOMException polyfills
├── types/                                # TypeScript Interfaces
│   ├── auth.ts                           # Auth and Session structures
│   └── property.ts                       # Property models
└── app.config.js                         # Dynamic Expo configuration
```

6.2 Routing & Navigation

- Expo Router (file-based navigation).
- Native bottom navigation bar with icons (*Home, Map, Add, Saved, Profile*).

- Responsive top navigation header bar (`WebTopNav`) rendering pages within a `Slot` view.

Auth Guard Logic

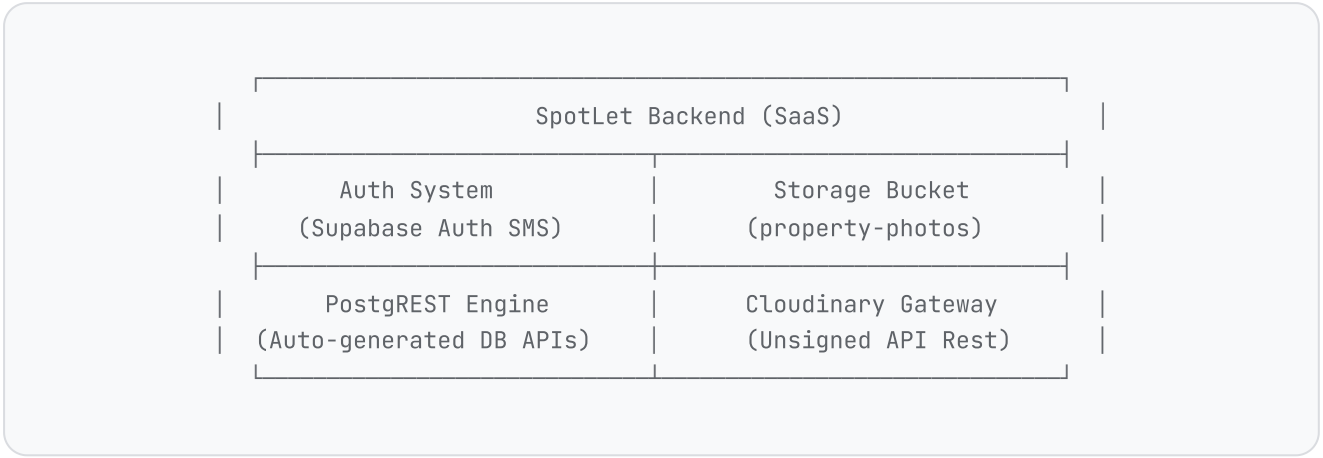
```typescript
const inAuthGroup = segments[0] === 'auth';
if (!isLoggedIn && !inAuthGroup) {
  router.replace('/auth/login');
} else if (isLoggedIn && inAuthGroup) {
  router.replace('/(tabs)');
}
```

6.3 Platform-Specific Components (The Web Map Hack)

DESIGN DECISION

To support both Google Maps on native mobile and interactive maps on web without compilation crashes, SpotLet uses extension resolver files:

- `CustomMap.tsx` resolves on iOS/Android — imports and exports `react-native-maps` .
- `CustomMap.web.tsx` resolves on Web — mounts a native HTML `iframe` rendering a Leaflet Map with CartoDB Dark tiles. Coordinates are synced via `window.postMessage` .



7.1 Database Interface

Database interactions are handled via Supabase's auto-generated PostgREST interface. There is . CRUD operations map directly to table commands through `lib/supabase.ts`.

7.2 Storage Architecture

- Clients upload picked media directly to Cloudinary using an unsigned upload preset (`spotlet_upload`). Cloudinary serves media over its global CDN.
- `uploadPropertyImage` is defined to upload images directly to Supabase's `property-photos` storage bucket when native storage integrations are required.

08

Entity Relationship Diagram

users

PK id	uuid
phone	text
name	text
avatar_url	text
created_at	timestamptz
updated_at	timestamptz

properties

PK id	uuid
FK owner_id	uuid
title	text
description	text
rent	numeric
deposit	numeric
type	text (enum)
furnished	boolean
for_whom	text (enum)
latitude	double
longitude	double
address	text
photos	text[]
owner_phone	text
owner_name	text
amenities	text[]
available	boolean
created_at	timestamptz

saved_properties

PK id	uuid
FK user_id	uuid
FK property_id	uuid

```
saved_at            timestamptz
```

`users` ⟶ *owns* ⟶ `properties` | `users` ⟶ *saves* ⟶ `saved_properties` | `properties` ⟶ *saved_by* ⟶ `saved_properties`

8.1 SQL Schema

Users Table

```sql
-- Create custom profiles table
CREATE TABLE public.users (
  id         uuid REFERENCES auth.users(id) ON DELETE CASCADE PRIMARY KEY,
  phone      text,
  name       text,
  avatar_url text,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now()
);
```

Properties Table

```sql
-- Create properties table
CREATE TABLE public.properties (
  id          uuid DEFAULT gen_random_uuid() PRIMARY KEY,
  owner_id    uuid REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
  title       text NOT NULL,
  description text,
  rent        numeric NOT NULL,
  deposit     numeric NOT NULL,
  type        text NOT NULL CHECK (type IN ('1BHK', '2BHK', '3BHK', 'PG', 'Room', 'Independent House')),
  furnished   boolean DEFAULT false NOT NULL,
  for_whom    text NOT NULL CHECK (for_whom IN ('Family', 'Bachelor', 'Any')),
  latitude    double precision NOT NULL,
  longitude   double precision NOT NULL,
  address     text NOT NULL,
  photos      text[] DEFAULT '{}' NOT NULL,
  owner_phone text NOT NULL,
  owner_name  text,
  amenities   text[],
  available   boolean DEFAULT true NOT NULL,
  created_at  timestamptz DEFAULT now() NOT NULL
);
```

Saved Properties Table

```sql
-- Create saved properties join table
CREATE TABLE public.saved_properties (
  id          uuid DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id     uuid REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
  property_id uuid REFERENCES public.properties(id) ON DELETE CASCADE NOT NULL,
  saved_at    timestamptz DEFAULT now() NOT NULL,
  UNIQUE (user_id, property_id)
);
```

8.2 Row Level Security (RLS) Policies

Properties Table RLS

```sql
ALTER TABLE public.properties ENABLE ROW LEVEL SECURITY;

-- Anyone can view available properties
CREATE POLICY "Anyone can view properties"
  ON public.properties FOR SELECT
  USING (available = true);

-- Only authenticated users can insert with their own UID
CREATE POLICY "Owners can insert their properties"
  ON public.properties FOR INSERT
  WITH CHECK (auth.uid() = owner_id);

-- Only listing owners can modify listings
CREATE POLICY "Owners can update their properties"
  ON public.properties FOR UPDATE
  USING (auth.uid() = owner_id);
```

Saved Properties Table RLS

```sql
ALTER TABLE public.saved_properties ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can view their saved properties"
  ON public.saved_properties FOR SELECT
  USING (auth.uid() = user_id);

CREATE POLICY "Users can save properties"
  ON public.saved_properties FOR INSERT
  WITH CHECK (auth.uid() = user_id);

CREATE POLICY "Users can unsave properties"
  ON public.saved_properties FOR DELETE
  USING (auth.uid() = user_id);
```

Users Table RLS

```sql
ALTER TABLE public.users ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can view their own profile"
  ON public.users FOR SELECT
  USING (auth.uid() = id);

CREATE POLICY "Users can update their own profile"
  ON public.users FOR ALL
  USING (auth.uid() = id);
```

8.3 Recommended Database Indexes

```sql
CREATE INDEX idx_properties_available ON public.properties (available) WHERE available = true;
CREATE INDEX idx_properties_location ON public.properties (latitude, longitude);
CREATE INDEX idx_saved_properties_user ON public.saved_properties (user_id);
```

09

9.1 Phone OTP Flow Detail

```
[User App Input] ── Phone number ──→ [Supabase Auth Client] ──→ [SMS Gateway] ──→ [User Device (OTP)]
[User App Input] ── Verify Code ──→ [Supabase Auth Client] ──→ [JWT Token Issued] ──→ [App Session Start]
```

1. Standardizes phone inputs on the client before request triggers.
2. Secure token exchange generating Access JWT and Refresh Token.
3. Encrypted auth state payloads stored in client's keychain or secure device storage.

9.2 OAuth Google Flow

- Custom deep-link: `spotlet://auth/callback`
- Uses `expo-web-browser` via `WebBrowser.openAuthSessionAsync(url, redirectUrl)`
- Parses implicit URL hashes directly if PKCE exchange code is missing.

9.3 Security Best Practices

SECURITY RULES

- Row-Level Enforcement: RLS policies prevent accessing or modifying other users' records.
- Client Env Hygiene: Only `EXPO_PUBLIC_` prefixed variables compiled in app packages. No service role keys shipped.
- Input Sanitization: Regex cleaning on phone numbers (+91 prefix validation) and descriptions.

10

10.1 Color Palette (Material Design 3 Dark)

Background #121212	Surface #1E1E1E	Primary #BB86FC
Secondary #03DAC6	Tertiary #CF6679	Border #2D2D2D
Text #FFFFFF	Text Sub #888888	

10.2 Typography

Element	Font	Size	Weight
Titles	Inter / Roboto	24–28px	Bold (700)
Cards	Inter / Roboto	16–18px	Semi-Bold (600)
Details / Pills	Inter / Roboto	12–13px	Medium / Bold
Descriptions	Inter / Roboto	13–14px	Regular (400)

10.3 Micro-Animations & Transitions

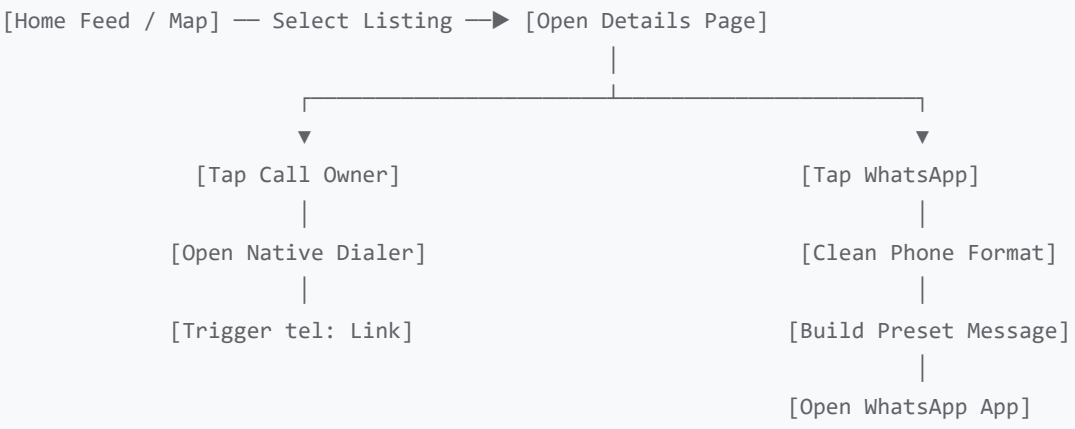
- Momentum-snapping intervals to focus cards smoothly as user swipes.
- Modals use `animationType="slide"` on mobile viewports.
- Semi-transparent card with blur filter for depth.

```css
backdrop-filter: blur(20px);
background-color: rgba(255, 255, 255, 0.04);
border: 1px solid rgba(255, 255, 255, 0.08);
```

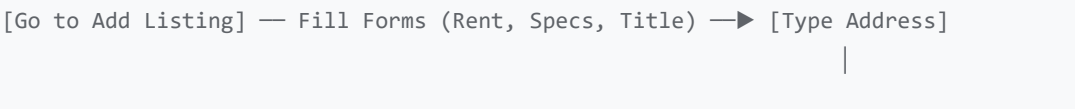
11

11.1 Onboarding & Phone Verification

11.2 Discover & Direct Contact



11.3 Property Creation




[Debounce Geocoder 1.5s]
[Update Coordinate Pin]
[Upload Media Gallery]
[Select Image Files]
[Upload to Cloudinary CDN]
[Submit Insert SQL]
[Reset Form & Redirect]

12.1 Supabase PostgREST Endpoint

BASE CONFIGURATION

Base URL: `https://<project-ref>.supabase.co/rest/v1`

Authorization: Bearer token (JWT User Session) + API Gateway Anon Key

12.2 Cloudinary REST Image API

Parameter	Details
	`POST https://api.cloudinary.com/v1_1/${CLOUD_NAME}/image/upload`
	`multipart/form-data`
	Image binary / blob
	Unsigned preset value (e.g., `spotlet_upload`)

Success Response

JSON

```
{
  "secure_url": "https://res.cloudinary.com/.../image.jpg",
  "public_id": "image_public_id",
  "format": "jpg"
}
```

12.3 Deep Link Integrations

Channel	URI Scheme	Notes
	`whatsapp://send?phone=${cleanPhone}&text=${message}`	Web fallback: `https://wa.me/${cleanPhone}?text=${message}`
	`tel:${owner_phone}`	Opens native dialer

WHATSAPP TEMPLATE

```
Hi, I'm interested in your property listing "${title}" on SpotLet.
```

13

13.1 Global State (AuthContext)

Property	Type	Purpose
`user`	AuthUser	User session profile information
`session`	Session	Active JWT session token
`isLoading`	boolean	Controls app boot splash rendering
`error`	string | null	Authentication feedback error messages

SAFETY NET

A 5-second timer forces `isLoading` to `false` on app boot if Supabase connection fails, preventing infinite loading states.

13.2 Local State & UI Caching

- Managed locally using React state, filtering cached listing arrays on the client side without triggering new database queries.
- Updates the local set of saved properties instantly. If the async API call fails, reverts the bookmark state to ensure smooth UI behavior.

14

14.1 Frontend Optimization

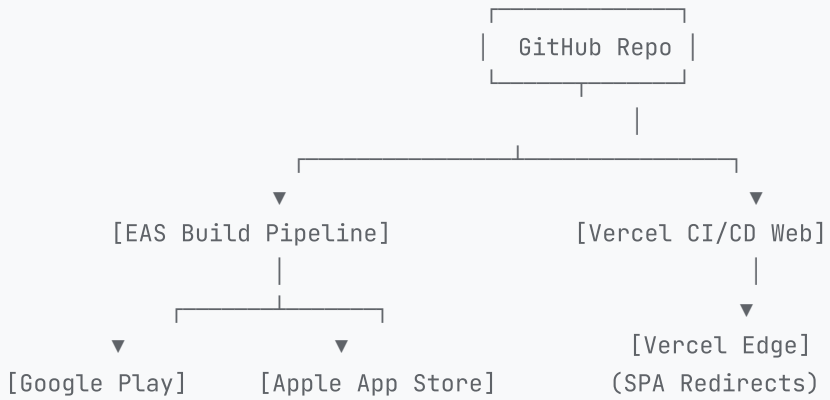
- 1.5-second debounce on property listing form. Queries geographical coordinates only after user stops typing.
- Platform-specific map wrappers prevent compilation crashes on web by replacing native Google Maps imports with Leaflet.
- Horizontal scroll lists snap to calculated indices on scroll events to prevent map UI lag.

14.2 Media Optimization

- Quality constraints (`quality: 0.8`) during media library selections.
- Unsplash placeholder assets on upload timeout failures.

15

15.1 Deployment Architecture



15.2 Web Deployment (Vercel SPA)

Web applications deploy on Vercel as single-page applications. Rewrites route all requests back to `/index.html`.

```json
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

`expo export --platform web` (outputs to `/dist`).

15.3 Mobile App Builds

- Handles native compilation of APK (Android) and IPA (iOS) binaries.
- Expo Updates to push updates directly to active user devices.

16.1 Testing Frameworks

Tool	Purpose	Command
Jest	Unit testing for utility & validation logic	`npm run test`
ESLint	Code style and quality checks	`npm run lint`

16.2 Verification Plan

- Run locally on iOS Simulator and Android Emulator, test on physical devices using Expo Go.
- Simulate offline scenarios to verify geocoder fallbacks and user error messaging.

17.1 Technical Limitations

⚠ DOMEXCEPTION CRASH ON HERMES

The Hermes engine on native Android/iOS does not include `DOMException`, causing Supabase client crashes on network errors.

Solution: Integrated a global polyfill in `lib/polyfills.ts` and `entry.js` to define `DOMException` on boot.

⚠ WEB COMPILATION FAILURES

`react-native-maps` cannot compile on web platforms.

Solution: Platform-specific map implementations (`CustomMap.tsx` and `CustomMap.web.tsx`) isolate Leaflet and native code respectively.

⚠ LEAFLET WEB MAP LIMITATIONS

Leaflet uses standard iframe container boundaries, limiting direct UI styling from React Native contexts.

17.2 Scalability & Security Risks

SMS RATE LIMITING

High traffic can quickly exhaust SMS validation budgets.

Mitigation: Enable Google OAuth as alternative, configure custom Twilio gateways with captcha checks.

CLIENT API KEY EXPOSURE

Expo bundles `EXPO_PUBLIC_` variables directly into client apps.

Mitigation: Strict database RLS policies. Anonymous key should only grant access within RLS parameters.

18.1 Short-Term Goals

Immediate

- Replace inputs with a visual range selector.
- Configure EAS pipelines for automated builds.
- Display average rental price densities.

18.2 Medium-Term Goals

Q3–Q4

- Messaging system to keep communication within the platform.
- Moderation interfaces and landlord document verification (Aadhaar).

18.3 Long-Term Goals

Future

- Personalized discovery feeds based on saved properties and search criteria.
- Integrated digital lease contracts and secure rental payment gateways.

OPTIMIZED FOR LLM INTERPRETERS

This section provides a machine-readable context block for AI agents continuing development on SpotLet.

YAML

```yaml
context:
  app_type: Cross-Platform Map-Based Rental Discovery App (Expo Router, Expo SDK 54)
  target_audience: India Residential Rental Market (BHKs, Rooms, PGs)
  accent_colors:
    primary: "#BB86FC"      # Vibrant Purple
    secondary: "#03DAC6"    # Teal
    background: "#121212"   # Pure dark grey
    surface: "#1E1E1E"      # Light grey
  critical_dependencies:
    supabase_auth: Phone SMS OTP passwordless flow & Google OAuth
    database: PostgreSQL schemas with active Row Level Security (RLS) policies
    maps_native: react-native-maps (compiles on mobile)
    maps_web: Leaflet iframe fallback inside CustomMap.web.tsx
    media: Cloudinary REST uploads via FormData

implementation_rules:
  1: All polyfills must compile before Expo entry imports. Use entry.js as main file.
  2: Do NOT import react-native-maps in web-resolved components. Use CustomMap wrapper.
  3: Stick to forced UserInterfaceStyle="dark" using react-native-paper MD3 dark themes.
  4: Do not bypass RLS policies on tables properties, saved_properties, or users.
  5: Deep links to telephone call and WhatsApp must use cleaned number patterns.

development_priorities:
  first: Push notification pipelines
  second: Visual price slider ranges
  third: Landlord document checks (Aadhaar verification)
```